

Correction détaillée du TD

Machines de Turing • Cubic • Pokémon • Phutball

Basé sur le PDF fourni. Quand une figure est un peu floue, j'explique surtout la logique du gadget et je signale l'hypothèse utilisée.

1. TD1 Machines de Turing

Énoncé lu sur les pages 1-2 : la machine a un ruban d'entrée en lecture seule et un ruban de travail lecture/écriture. Les états utiles sont q_0 , q_1 , q_2 , q_a , q_r .

Exercice 1

Idée globale avant de dérouler les calculs :

- q_0 copie le mot du ruban d'entrée vers le ruban de travail, symbole par symbole, en avançant sur les deux rubans.
- q_1 rembobine la tête du ruban d'entrée vers la gauche jusqu'au début du mot, tandis que la tête du ruban de travail reste à la fin du mot copié.
- q_2 compare le mot d'entrée de gauche à droite avec sa copie de droite à gauche. Donc la machine teste si le mot est un palindrome.

Je note un mot $w = a_1a_2\dots a_n$.

Question 1 — Exécution sur 01110 et 11010111

Entrée 01110 :

- En q_0 , la machine recopie 0, puis 1, puis 1, puis 1, puis 0 sur le ruban de travail. À la fin, le ruban de travail contient 01110 et les deux têtes sont juste après la fin du mot.
- En q_1 , la tête du ruban d'entrée repart à gauche jusqu'au blanc précédant le mot. Ensuite la transition $(q_1, \#, \#) \rightarrow q_2$ place la tête du ruban d'entrée sur le premier symbole et la tête du ruban de travail sur le dernier symbole du mot copié.
- En q_2 , on compare successivement : 0 avec 0, puis 1 avec 1, puis 1 avec 1, puis 1 avec 1, puis 0 avec 0.
- On atteint ensuite simultanément les deux blancs, donc la transition vers q_a s'applique.

Conclusion : 01110 est accepté.

Entrée 11010111 :

- Après la phase q_0 , le ruban de travail contient 11010111.
- Après le retour en q_1 , q_2 compare le premier symbole du mot avec le dernier, puis le deuxième avec l'avant-dernier, etc.
- Première comparaison : 1 avec 1, c'est bon.
- Deuxième comparaison : 1 avec 1, c'est encore bon.
- Troisième comparaison : 0 avec 1, désaccord.
- La transition de rejet s'applique : la machine va vers q_r .

Conclusion : 11010111 est rejeté.

Vérification conceptuelle : 01110 est bien un palindrome, alors que 11010111 ne l'est pas.

Question 2 — Rôle de chaque état

- q_0 : état de copie. Il lit le mot d'entrée et le recopie à l'identique sur le ruban de travail.
- q_1 : état de retour arrière sur le ruban d'entrée. Il replace la tête au début du mot pour préparer la comparaison.
- q_2 : état de comparaison symétrique. Il lit l'entrée de gauche à droite et la copie de droite à gauche.
- q_a : état d'acceptation, atteint si toutes les comparaisons réussissent et si l'on arrive au blanc en même temps sur les deux rubans.
- q_r : état de rejet, atteint dès qu'on rencontre deux symboles différents pendant la phase de comparaison.

Question 3 — Langage reconnu

Le langage reconnu est l'ensemble des palindromes binaires :

$$L = \{ w \in \{0,1\}^* \mid w = w^R \}$$

où w^R désigne le mot renversé.

Justification : la machine compare le i -ième symbole du mot avec le i -ième symbole en partant de la fin. Elle accepte exactement quand tous les couples coïncident.

Question 4 — Complexité

- Temps : $O(n)$.
- Explication : il y a une première passe de copie en q_0 (n étapes), une passe de retour en q_1 (n étapes environ), puis une passe de comparaison en q_2 (n étapes environ, ou $n/2$ comparaisons conceptuelles mais une transition par symbole). Au total, c'est linéaire.
- Espace de travail : $O(n)$.
- Explication : le ruban de travail stocke une copie complète du mot, donc il utilise n cases utiles à constante près.

Exercice 2 — Construire une machine qui écrit n^2 bits puis 2^n bits

Ici, le plus propre est de donner une construction de haut niveau. En théorie des automates, c'est totalement acceptable tant qu'on précise clairement les registres/counters et la logique.

1) Écrire n^2 bits sur le ruban de travail

Construction simple :

- Étape 1 : parcourir l'entrée une fois pour compter sa longueur n . On peut stocker n en unaire sur une zone auxiliaire du ruban de travail.
- Étape 2 : construire deux compteurs unaires de taille n : un compteur externe et un compteur interne.
- Étape 3 : pour chaque marque du compteur externe, parcourir entièrement le compteur interne et écrire un bit (par exemple 1) sur une nouvelle zone du ruban.
- Chaque itération externe déclenche n écritures, et il y a n itérations externes : on écrit donc $n \times n = n^2$ bits.

Version encore plus concrète : si l'entrée contient n symboles, on recopie d'abord n jetons 'X'. Puis pour chaque X de la première copie, on refait un parcours complet de la seconde copie et on écrit un 1 à chaque passage.

Le nombre de bits écrits est exactement n^2 .

2) Écrire 2^n bits sur le ruban de travail

Une construction standard consiste à énumérer tous les mots binaires de longueur n .

- Étape 1 : compter n à partir de l'entrée.
- Étape 2 : créer sur le ruban de travail un compteur binaire de n bits initialisé à $00\dots 0$.
- Étape 3 : à chaque valeur du compteur, écrire un bit de sortie, par exemple 1.
- Étape 4 : incrémenter le compteur et recommencer jusqu'à $11\dots 1$ inclus.

Comme il existe exactement 2^n valeurs possibles pour un compteur binaire de n bits, la machine effectue exactement 2^n itérations et écrit donc 2^n bits.

Remarque utile : on pourrait aussi produire 2^n bits par doublement successif, en partant d'un bit et en recopiant tout le bloc déjà écrit n fois. Après k étapes on a 2^k bits, donc après n étapes on a 2^n bits.

Exercice 3 — Ruban de travail infini dans un seul sens

But à montrer : toute machine tournant en temps $T(n)$ avec ruban bi-infini peut être simulée par une machine dont le ruban de travail n'est infini que vers la droite, avec un surcoût constant, ici borné par $4T(n)$.

Idée classique : on code les positions entières du ruban bi-infini dans les entiers naturels par entrelacement :

$0 \mapsto 0, 1 \mapsto 2, 2 \mapsto 4, \dots$ et $-1 \mapsto 1, -2 \mapsto 3, -3 \mapsto 5, \dots$

Autrement dit, les cases non négatives vont sur les positions paires, et les cases négatives sur les positions impaires.

- Si la machine simulée est sur la case $i \geq 0$, on place la tête simulatrice sur $2i$.
- Si elle est sur la case $-i$ avec $i \geq 1$, on la place sur $2i-1$.

Pourquoi cela marche :

- Un déplacement d'une case vers la droite sur Z devient soit un déplacement de 2 cases sur N , soit un petit changement de parité avec au plus 2 mouvements.
- Un déplacement d'une case vers la gauche sur Z se traite de la même façon.
- La lecture/écriture du symbole reste locale : une fois la case codée atteinte, on lit ou on écrit le symbole simulé.

Donc chaque transition de la machine originale peut être simulée par un nombre constant d'actions élémentaires. Une borne grossière classique est 4 opérations pour 1 opération simulée.

Conclusion : si la machine originale tourne en temps $T(n)$, la machine à ruban unilatéral tourne en temps $O(T(n))$, et même $\leq 4T(n)$ avec ce codage.

2. Cubic

Énoncé lu sur les pages 3-4. Le but est de faire disparaître tous les blocs colorés. Deux blocs de même couleur s'annulent quand ils deviennent adjacents par un côté, puis les blocs du dessus tombent. Le reste du TD construit une réduction depuis SAT.

Question 1 — Résoudre les deux petits jeux de la figure 1

La figure est un peu floue, donc je donne surtout la logique de résolution.

Jeu 1 :

- Le couloir vertical central permet de faire tomber les deux blocs '1' l'un vers l'autre, puis les deux blocs '2' l'un vers l'autre.
- Une stratégie naturelle consiste à décaler d'abord l'un des blocs 1 pour provoquer l'adjacence, ce qui vide une partie de la colonne, puis à exploiter la chute résultante pour aligner les deux blocs 2.

Conclusion : le jeu 1 est soluble.

Jeu 2 :

- Le gadget semble conçu comme une file de paires à éliminer dans l'ordre 1,2,3,4,5.
- Le long couloir blanc sert à amener un bloc de la rangée du haut vers la zone où il rencontre son double dans la rangée du bas.
- Chaque disparition libère l'espace pour la suivante, ce qui correspond déjà à l'idée de propagation utilisée dans les gadgets logiques.

Conclusion : le jeu 2 est également soluble, par éliminations successives des paires.

Question 2 — SAT sous forme de circuit

On transforme une formule booléenne en circuit orienté acyclique :

- Les feuilles sont les variables $x_1, \dots, x_n$ et éventuellement leurs négations.
- Les nœuds internes sont des portes logiques AND, OR, NOT.
- Tous les arcs vont du haut vers le bas, donc on calcule la valeur par propagation.
- La formule est satisfiable s'il existe une affectation des variables qui rend la sortie du circuit vraie.

Cette version 'circuit SAT' est pratique parce que le TD construit ensuite des gadgets Cubic pour simuler les portes et les fils.

Question 3 — Encodage des variables et de la sortie

L'encodage d'une variable force un choix binaire : une configuration correspond à 'mettre la variable à vrai', l'autre à 'la mettre à faux'.

- Une fois le choix fait dans le gadget variable, un seul des deux chemins logiques reste exploitable en aval.
- Ce chemin représente soit le littéral x , soit le littéral $\neg x$.
- La pièce 'sortie' est construite pour être vidable si et seulement si le signal logique reçu vaut VRAI.

Autres gadgets nécessaires :

- des fils pour transporter l'information,
- une porte AND,
- une porte OR,
- une porte NOT,
- un gadget SWITCH / aiguillage pour router le signal,
- un croisement de fils pour éviter de tout rendre planaire de façon artificielle.

Question 4 — Fonctionnement de la porte AND

Principe logique attendu : la sortie doit être libérée si et seulement si les deux entrées valent 1.

Dans un gadget Cubic, cela se fait en imposant deux déblocages indépendants :

- si une seule entrée est activée, une partie du couloir reste bloquée, donc on ne peut pas finir de vider la structure ;
- si les deux entrées sont activées, chacune retire un bloc-clef, les chutes se combinent, et la sortie devient accessible.

Donc le gadget implémente bien $A \wedge B$.

Question 5 — Construire NOT et OR

NOT :

- On inverse la convention de présence/absence du signal.
- Par exemple, si 'entrée vraie' signifie 'un couloir est ouvert', alors la porte NOT est conçue pour ouvrir la sortie précisément quand ce couloir d'entrée n'a pas été ouvert.
- Techniquement, on peut utiliser un bloc-tampon qui tombe et bloque la sortie quand l'entrée est vraie, et qui laisse passer sinon.

OR :

- On veut que la sortie soit libérée dès qu'au moins une des deux entrées l'est.
- On met donc deux mécanismes alternatifs de déblocage menant à la même sortie.
- Si A est vraie, le premier chemin suffit ; si B est vraie, le second suffit ; si A et B sont vraies, cela marche aussi ; si les deux sont fausses, rien ne se débloque.

On obtient ainsi $A \vee B$.

Question 6 — Rôle du SWITCH

Le gadget SWITCH sert d'aiguillage : il redirige un signal vers une sortie choisie, ou il permet de faire dépendre un passage d'un état préalable du gadget.

Dans une réduction logique, ce type de pièce est utile pour :

- séparer des fils,
- imposer un ordre de passage,
- réaliser des croisements contrôlés en combinant plusieurs switches.

Autrement dit, ce n'est pas une porte logique de base, mais un gadget structurel.

Question 7 — Construire un croisement avec des SWITCH

Idée : on remplace un croisement direct de deux fils x et y par une petite composition de SWITCH qui garantit :

- le signal x ressort sur x' sans perturber y,
- le signal y ressort sur y' sans perturber x,
- l'ordre de passage éventuel ne détruit pas l'autre fil.

En pratique, on fait entrer chaque fil dans un aiguillage qui réserve temporairement l'espace nécessaire, puis on rétablit l'autre canal. C'est exactement le rôle des gadgets de croisement dans les preuves de NP-complétude à gadgets.

Question 8 — Construction mystère

La pièce 'mystère' ressemble à un gadget composé servant de croisement conditionnel ou de permutation de fils.

Vu son placement à côté du SWITCH dans la planche des gadgets, l'interprétation la plus cohérente est :

- elle permet à deux signaux de traverser la zone sans se gêner,
- ou elle transforme une entrée haute/basse en une sortie réordonnée.

Autrement dit, c'est très probablement un gadget de routage plus élaboré qu'une simple porte logique.

Question 9 — Montrer que Cubic est NP-complet

Schéma de preuve standard :

- Cubic est dans NP : un certificat est une suite de coups. On simule les déplacements et les chutes en temps polynomial et on vérifie qu'à la fin tous les blocs ont disparu.
- Cubic est NP-dur : on part d'une instance de SAT (ou Circuit-SAT).
- On remplace chaque variable par un gadget variable qui force un choix vrai/faux.
- On relie les variables aux clauses via des fils et des gadgets de croisement.
- On implémente les opérations logiques avec AND/OR/NOT.
- La pièce de sortie est soluble si et seulement si la formule est satisfaite.

Donc l'instance Cubic est soluble si et seulement si la formule initiale est satisfiable. La réduction est polynomiale, donc Cubic est NP-complet.

3. TD NP-complétude — Pokémon

Énoncé lu sur les pages 5–8. Ici le héros traverse des cases blanches ; les arbustes noirs sont infranchissables ; les dresseurs déclenchent un combat. Les dresseurs 'débutants' peuvent être battus et deviennent alors des obstacles inactifs ; les professionnels donnent un 'game over'.

Question 1 — Traverser le labyrinthe de la figure 3

La logique générale du labyrinthe est de forcer le joueur à utiliser les dresseurs débutants comme mécanismes de déblocage tout en évitant les professionnels.

Une stratégie sûre consiste à :

- attirer un dresseur débutant quand cela ouvre un couloir,
- utiliser ensuite sa case comme nouvel obstacle fixe,
- ne jamais entrer dans le champ de vision d'un professionnel.

Comme la figure est scannée et peu nette, je ne donne pas une suite de cases exacte ; l'objectif du TD est surtout de comprendre la fonction des pièces qui servent à la réduction.

Question 2 — Concevoir la pièce 'variable'

La pièce variable doit forcer un choix exclusif entre deux chemins :

- un chemin représente la mise à vrai de x ;
- l'autre représente la mise à faux de x .
- Une fois le héros engagé dans l'un des deux sous-couloirs, la géométrie de la pièce et le comportement des dresseurs doivent empêcher de revenir prendre l'autre branche.

Cette irréversibilité est essentielle : elle correspond au fait qu'une variable reçoit une seule valeur dans une affectation.

Question 3 — Concevoir la pièce 'sens unique'

La pièce sens unique doit autoriser le passage dans un sens mais le rendre impossible ou mortel dans l'autre.

- Version typique : dans le sens autorisé, on déclenche un dresseur débutant de sorte qu'après le combat sa position bloque le retour.
- Dans le sens interdit, soit on se retrouve face à un professionnel, soit on active un combat qui rend la traversée impossible.

Le but est exactement celui d'une diode logique sur un graphe de réduction.

Question 4 — Rôle de la pièce 'clause' (figure 4)

La pièce clause représente une clause à trois littéraux.

- Chaque entrée possible vers la pièce correspond à un littéral de la clause.
- Si au moins une des trois entrées est atteinte depuis les pièces variables, le héros peut neutraliser un dresseur débutant clé dans la pièce.
- Ce déblocage rend ensuite possible la traversée horizontale finale de la clause.

Donc la pièce clause est franchissable à la fin si et seulement si la clause a été 'satisfaite' par au moins un littéral vrai.

Question 5 — Rôle de la figure 5

La figure 5 est un gadget de croisement de fils.

Les entrées x et y doivent pouvoir ressortir respectivement en x' et y' sans interférence logique.

- Un parcours venant de x doit mener à x' sans ouvrir indûment le canal $y \rightarrow y'$.
- Un parcours venant de y doit mener à y' sans ouvrir indûment le canal $x \rightarrow x'$.

C'est indispensable pour transporter plusieurs littéraux dans un plan sans casser la réduction.

Question 6 — Montrer que Pokémon est NP-complet

Le PDF contient déjà une réponse sur la page 8 ; je la reformule de façon plus pédagogique.

- Pokémon $\in$ NP : un certificat est un chemin du héros du départ vers l'arrivée. On peut simuler les déplacements, les déclenchements de combats et l'effet permanent des dresseurs battus en temps polynomial.
- NP-difficulté : réduction depuis 3-SAT.
- Chaque variable est remplacée par une pièce variable imposant un choix vrai/faux.
- Chaque clause est remplacée par une pièce clause que l'on peut débloquent si l'un de ses trois littéraux est choisi.

- On ajoute des pièces sens unique et des croisements pour router les connexions.

Correction logique :

- Si la formule ϕ est satisfiable, on suit dans chaque pièce variable le chemin correspondant à l'affectation satisfaisante ; chaque clause a alors au moins un littéral vrai, donc chaque pièce clause peut être débloquée ; le héros atteint la sortie.
- Réciproquement, si le héros atteint la sortie, alors il a nécessairement traversé toutes les pièces clause après les avoir débloquées ; cela signifie que pour chaque clause au moins un littéral correspondant a été activé dans une pièce variable ; on en déduit une affectation satisfaisante.

Donc Pokémon est NP-complet.

4. TD Phutball

Énoncé lu sur les pages 9–11. Le ballon se déplace en sautant horizontalement ou verticalement au-dessus d'une suite contiguë de joueurs blancs, qui sont ensuite retirés. Tant qu'un nouveau saut est possible depuis la case d'arrivée, le même joueur peut continuer.

Exercice 1

Question 1 — Position de la figure 2b

L'idée stratégique est que le joueur 1 veut préparer une chaîne de sauts menant à la sortie par la gauche, tandis que le joueur 2 cherche à casser cette chaîne ou à se ménager une sortie par la droite.

Comme la figure est peu lisible, le plus important est le raisonnement :

- un coup de Phutball peut être très long car un seul tour peut enchaîner plusieurs sauts ;
- placer un joueur blanc ne sert pas seulement à préparer son propre futur saut, mais aussi à empêcher certaines lignes de saut adverses ;
- la 'bonne' stratégie en un petit nombre de coups consiste souvent à créer une menace de victoire forcée sur une ligne, puis à profiter du fait que la défense ouvre une autre ligne.

Sans lecture parfaite des coordonnées sur le scan, je n'affirme pas ici une suite exacte de 6 coups case par case. En revanche, c'est bien ce type d'argument de menace forcée que l'exercice vise.

Question 2 — Borne sur le nombre de coups possibles en un tour

Sur un plateau $n \times n$, il y a $(n+1)^2$ intersections si l'on raisonne comme sur une grille de Go carrée, mais une borne simple suffit.

- Action 1 : poser un joueur. Il y a au plus $O(n^2)$ positions possibles.
- Action 2 : déplacer le ballon. À chaque étape d'un tour, le ballon choisit une direction parmi 4, puis saute une suite contiguë de joueurs blancs.
- Chaque saut retire au moins un joueur blanc. Donc, au cours d'un tour, le nombre total de sauts est au plus le nombre de joueurs présents, donc $O(n^2)$.
- À chaque étape il y a au plus 4 choix de direction, donc une borne très grossière sur le nombre de séquences de sauts est $4^{O(n^2)}$.

Si l'on demande juste une borne supérieure, on peut répondre : le nombre de coups possibles en un tour est exponentiel dans n^2 . C'est précisément ce qui rend le jeu délicat du point de vue algorithmique.

Question 3 — Montrer que déterminer si une position est gagnante est dans NP

Un certificat naturel est une séquence de sauts constituant un coup gagnant immédiat.

- On vérifie en temps polynomial que chaque saut est légal : direction horizontale ou verticale, présence d'au moins un joueur adjacent dans cette direction, saut jusqu'à la première intersection libre, suppression des joueurs sautés.
- On vérifie ensuite que la position finale fait sortir le ballon du bon côté.

Comme la longueur d'une séquence de sauts est au plus le nombre de joueurs présents, donc polynomialement bornée par la taille de l'instance, la vérification est polynomiale. Donc le problème est dans NP.

Exercice 2 — Gadgets

Question 1 — 3-sélecteur vertical

Le gadget 3a permet d'entrer par une entrée et de ressortir par l'une de trois sorties verticales possibles.

- La disposition des joueurs blancs force un choix exclusif de trajectoire.
- Après un premier passage, les joueurs sautés disparaissent ; le gadget change donc d'état.

C'est utile pour modéliser une clause à trois littéraux : le ballon doit pouvoir choisir l'un des trois canaux.

Question 2 — Croisement

Le gadget 3b permet à deux 'fils' de se croiser sans se perturber.

- Un premier passage consomme certains joueurs du gadget.
- La question importante est donc : reste-t-il un second passage indépendant ?

Dans un bon gadget de croisement, oui, mais seulement de manière contrôlée : le passage sur un canal ne doit pas ouvrir arbitrairement l'autre. L'objectif est exactement de préserver l'indépendance des deux connexions logiques.

Question 3 — Fusible

Le gadget 3c sert à empêcher un double usage.

- Après un premier passage, les joueurs clés sont consommés.
- Le gadget ne peut alors plus être retransversé dans les mêmes conditions.

Il joue donc le rôle d'une ressource à usage unique, d'où le nom de fusible.

Exercice 3 — NP-complétude de Phutball

Le texte des pages 10-11 donne déjà l'architecture de la réduction depuis 3-SAT.

- Pour chaque variable v , on crée deux lignes horizontales de joueurs : l'une pour v , l'autre pour $\neg v$.
- Les deux lignes sont reliées par un 2-sélecteur horizontal : le ballon devra choisir l'une des deux, ce qui encode la valeur de la variable.
- Pour chaque clause, on crée trois colonnes verticales correspondant aux trois littéraux de la clause.

- Les extrémités de ces colonnes sont reliées par un 3-sélecteur vertical : cela code le fait qu'il suffit d'un littéral vrai pour satisfaire la clause.
- Quand une colonne de clause croise la ligne d'un littéral incompatible, on place un fusible.
- Tous les autres croisements sont de simples gadgets de croisement.

Pourquoi la réduction marche :

- Choisir une ligne variable revient à fixer vrai ou faux.
- Une clause est franchissable si au moins une de ses trois colonnes reste compatible avec les choix faits sur les variables.
- Le ballon atteint la sortie si et seulement si toutes les clauses sont satisfaites.

Donc décider si la position est gagnante est NP-difficile. Comme on a déjà vu que le problème est dans NP, on obtient que le problème est NP-complet.

Remarques finales

- La partie la plus 'solide' du corrigé concerne les questions théoriques : langage reconnu, complexités, rôle des gadgets, schéma des réductions.
- Pour les questions très visuelles dépendant d'un chemin exact dans une figure scannée et légèrement floue, j'ai privilégié l'explication du mécanisme plutôt qu'un faux détail arbitraire.
- Si tu veux, je peux ensuite te faire une version 2 encore plus scolaire, exercice par exercice, avec davantage de pseudo-traces et de dessins simplifiés.